Propuesta de eficiencia — consumo de Anthropic + OpenAI TTS

Fecha: 2026-04-22 · Proyecto: Intrale Platform · Pipeline V2 · Autor: análisis automático Claude Code

Resumen ejecutivo

Hallazgo crítico #1 — no hay observabilidad real de tokens.
El campo `tokens_input/output/total` en `agent-metrics-history.jsonl` está en `null` en el 100% de las sesiones. Solo existe `tokens_estimated` (heurística, no real). **No podés optimizar lo que no medís** → precondición para cualquier ahorro.

Hallazgo crítico #2 — prompt caching desactivado.
Cada agente lanzado envía el rol completo (4-12 KB) sin `cache_control`. El rol base (`_base.md` , 3483 bytes ≈ 870 tokens) y el rol específico (hasta 11.750 bytes ≈ 2.938 tokens para QA) se facturan como input fresh en cada invocación. **Ahorro estimado activando caching: 40-70% en tokens de input**.

Hallazgo crítico #3 — OpenAI TTS sin alternativa gratuita activa.
Todo TTS usa `gpt-4o-mini-tts` con voz "ash". Edge-TTS (gratis, misma calidad para español) fue evaluada pero nunca activada en producción. Los textos de `rejection reports` (120-150 chars) + voice responses de Telegram son volumen bajo pero recurrente. **Ahorro estimado usando edge-tts como primario: 100% del costo TTS**.

1. Anthropic — análisis del consumo

1.1 Roles y tamaños

El pipeline tiene **16 roles** (`.pipeline/roles/*.md`) que se envían como system prompt en cada spawn. Tamaños:

Rol	Bytes	Tokens est.	Frecuencia de uso
`_base.md` (siempre)	3.483	~870	100% de los lanzamientos
`qa.md`	11.750	~2.938	1 por verificación
`build.md`	8.327	~2.082	1 por build
`po.md`	8.153	~2.038	validación + aprobación
`ux.md`	5.913	~1.478	validación + aprobación
`pipeline-dev.md`	5.541	~1.386	dev pipeline
resto (10 archivos)	14.876	~3.720 total	varios
Total roles	**58.043**	**~14.511**	

Peor caso por lanzamiento: 3.808 tokens solo en system prompt (`_base + qa`).

1.2 Cómo se spawnean los agentes

Desde `.pipeline/pulpo.js:4088, 4130-4140` :

```
const args = ['-p', userPrompt, '--system-prompt-file', systemFile,
              '--output-format', 'stream-json', '--verbose',
              '--permission-mode', 'bypassPermissions'];
```

Sin especificación de modelo (usa default), **sin flags de caching** (ni `--cache-control` , ni marcadores de cache-breakpoint). Cada lanzamiento paga el 100% del input.

1.3 Patrones ineficientes identificados

1. **YAML verbatim en el user prompt** (`pulpo.js:3954`): se inyecta el archivo de trabajo completo sin truncar. Issues con contexto largo (descripción + acceptance criteria + comments) cargan miles de tokens innecesarios.
2. **Rebotes sin compactación**: cada rebote (rechazo → dev → re-verificación) reinicia contexto con el motivo de rechazo verbatim + paths de logs + 7 pasos de instrucciones. Sin compendio iterativo.
3. **Sistema prompt archivado por cada lanzamiento** (`pulpo.js:3950-3951`): se genera `logs/agent-<issue>-<skill>-system.txt` fresh en cada spawn, sin reuso entre agentes del mismo skill.

4. **68 hooks en `.claude/hooks/`** (archivos `.js`). Desconocemos cuántos se ejecutan por mensaje — cada uno consume contexto. No hay trace ni budget.
5. **Sin budget por skill**: no hay límite superior de tokens por lanzamiento. Un agente en bucle de pensamiento puede generar miles de output tokens sin corte automático.

1.4 Métrica actual (muestra histórica)

Único reporte disponible — SPR-0051, del 2026-03-17 al 2026-03-22:

Skill	Sesiones	Tokens estimados	Avg/sesión	Min/sesión
Agente #722	1	317.125	317.125	183
/scrum,/planner	1	216.735	216.735	156
/ops	3	79.875	26.625	18
/po	2	21.330	10.665	4
ad-hoc	3	5.735	1.912	<1
Total	**12**	**654.410**		

Nota: estos son tokens *estimados*, no reales. Un agente largo (#722, 183 min) consumió 48% del total del sprint. Sugiere que sesiones largas son el driver principal.

2. OpenAI TTS — análisis del consumo

2.1 Dónde se invoca

Archivo	Línea	Uso
`.pipeline/multimedia.js`	226-270	función `textToSpeech()`
`.claude/hooks/commander/multimedia-handler.js`	177-215	función `callOpenAITTS()`
`.pipeline/rejection-report.js`	1544-1576	audio para cada rejection report

2.2 Configuración actual

- **Modelo**: `gpt-4o-mini-tts`
- **Voz**: `ash` con *instructions* porteño rioplatense
- **Fallback**: ElevenLabs (si hay `elevenlabsApiKey`)
- **Edge-TTS**: **NO implementado** (solo mencionado en sesiones históricas)
- **Formato**: `opus`

2.3 Cuándo dispara

- **Rejection reports**: cada vez que un agente rechaza en cualquier fase (análisis, dev, build, verificación, entrega). Texto 120-150 chars. Frecuencia ≈ 5-15/día en días normales.
- **Voice responses**: solo cuando un humano manda audio al bot. Bajo volumen.
- **NO dispara** en `/status`, heartbeats, `/help`.

2.4 Consumo estimado

Volumen diario estimado (conservador):

- Rejection reports: 10 × ~135 chars = ~1.350 chars/día
- Voice responses: ~3 × ~500 chars = ~1.500 chars/día
- **Total ≈ 3.000 chars/día**

Precio `gpt-4o-mini-tts`: **$0.015 / 1K chars** → **~$0.045/día** (~$1.35/mes). Bajo en términos absolutos, pero **100% evitable** con edge-tts.

3. Propuestas de mejora

Prioridad 1 — Quick wins (1-3 días)

> **P1.1 — Activar prompt caching de Anthropic**
> Agregar `cache_control: { type: "ephemeral" }` al bloque del system prompt cuando se use la Messages API. Si se usa el CLI `claude`, verificar si soporta `--cache-prompt` o similar; si no, migrar a invocación vía SDK.
>
> - **Impacto**: reduce costo de input tokens 40-70% para roles repetidos (90% de los spawns usan `_base.md`).
> - **Costo**: cambio chico en `pulpo.js` spawner, 2-3 líneas.
> - **Riesgo**: bajo — el caching es transparente.

P1.2 — Edge-TTS como primario, OpenAI TTS como fallback
Reordenar la cadena de fallback: **edge-tts** → **OpenAI** → **ElevenLabs**. Edge-tts es gratis, tiene voces en español (es-AR-ElenaNeural, es-AR-TomasNeural).

- **Impacto**: 100% del costo TTS eliminado (~$1.35/mes).
- **Costo**: instalar `edge-tts` npm package + wrapper en `multimedia.js`.
- **Riesgo**: bajo — OpenAI queda como fallback.

P1.3 — Tracking real de tokens
Capturar la respuesta del CLI `claude` (tiene JSON con `usage.input_tokens`, `cache_creation_input_tokens`, `cache_read_input_tokens`, `output_tokens`). Actualmente el Pulpo descarta esa info.

- **Impacto**: habilita todas las otras optimizaciones basadas en datos reales.
- **Costo**: parseo del stream-json en `pulpo.js` + escritura JSONL.
- **Riesgo**: ninguno.

Prioridad 2 — Optimizaciones estructurales (1-2 semanas)

P2.1 — Compactar roles redundantes
Los roles comparten mucho boilerplate (reglas de branch, PR conventions, QA gates) que también está en `CLAUDE.md`. Mover contenido duplicado a `_base.md`, dejar roles específicos con **solo lo propio de su skill**. Target: reducir roles mayores (qa.md: 11.7K → ~4K, build.md: 8.3K → ~3K).
Impacto: 30-50% menos tokens en system prompt por lanzamiento.

P2.2 — Truncar YAML del work file
En `pulpo.js:3954`, limitar el dump del YAML a campos esenciales (`issue, fase, pipeline, modo, motivo_rechazo_reciente`). Issues con 50+ comments inyectan 10-20K tokens que el agente no necesita.
Impacto: 20-40% menos tokens en user prompt.

P2.3 — Budget de tokens por skill
Agregar `max_tokens_per_agent` en `config.yaml` por skill. Agentes que llegan al límite → cierre automático con reporte. Evita sesiones runaway como la del agente

#722 (317K tokens en 183 min).
Impacto: techo duro al consumo; protege contra casos patológicos.

P2.4 — Dashboard de costo de tokens
Sumar panel al dashboard V2 con consumo diario por skill, top consumidores, tendencia semanal. Fuente: JSONL real del P1.3.
Impacto: visibilidad operativa continua; permite alertas automáticas.

Prioridad 3 — Optimizaciones avanzadas (2-4 semanas)

P3.1 — Ruteo por modelo según complejidad
Skills simples (tester verificación básica, security quick-scan, po acceptance check) pueden usar Haiku (cheaper/faster). Skills complejos (planner, pipeline-dev, build debug) usan Sonnet. Reservar Opus solo para casos excepcionales.
Impacto: 50-80% menos costo en skills "chicos" (que son el 60% de los lanzamientos).

P3.2 — Reuso de contexto en rebotes
En vez de reiniciar un agente dev tras rechazo, enviar un message adicional al mismo agente con `cache_control` del contexto previo. Requiere refactor de `pulpo.js` para mantener sesiones persistentes.
Impacto: 70-90% menos tokens en segundo/tercer intento.

P3.3 — Auditoría de 68 hooks
Revisar qué hooks corren por mensaje vs sesión. Deshabilitar los que no aportan valor en cada invocación. Algunos pueden ser *on-demand* en vez de *always-on*.
Impacto: reduce el overhead por cada interacción.

4. Roadmap propuesto

Fase	Ítems	Tiempo	Ahorro estimado
Sprint 1	P1.1, P1.2, P1.3	3-5 días	40-70% en input tokens + 100% TTS
Sprint 2	P2.1, P2.2, P2.3, P2.4	1-2 semanas	+20-40% adicional en tokens
Sprint 3	P3.1, P3.2, P3.3	2-4 semanas	+30-50% adicional según perfil de uso

Target total combinado: reducción 60-80% del consumo Anthropic y 100% del consumo OpenAI TTS, manteniendo la misma capacidad operativa del pipeline.

5. Precondiciones

1. **P1.3 (tracking real) es bloqueante** para medir si las demás optimizaciones funcionan. Hacer primero.
2. **Crear issues específicos** para cada propuesta (P1.1, P1.2, P1.3 como `priority:high`, resto como `priority:medium`).
3. **Verificar soporte de caching en el CLI** `claude`: si no lo soporta nativo, puede requerir migrar a SDK. El análisis de viabilidad va dentro de P1.1.

Generado automáticamente · Intrale Platform Pipeline V2 · 2026-04-22